

1. PROBLEM STATEMENT

To store a supermarket sales dataset in the Hadoop Distributed File System (HDFS) and to perform basic file-management and analysis operations on it using HDFS shell commands and Linux utilities.

The practical requires the student to:

- Verify the Hadoop installation and confirm that the HDFS services are running.
- Create the CSV file **supermarket_sales.csv** on the local file system.
- Create a directory **/supermarket** in HDFS and upload the CSV file into it.
- View the file contents, count the records, and search for a city (**Yangon**) and a product line (**Food and Beverages**).
- Calculate the total quantity sold using a command-line pipeline.
- Check the file size, make a backup copy inside HDFS, and download the file back to the local system.
- Stop the HDFS services after completing the work.

2. INTRODUCTION TO HADOOP

Apache Hadoop is a free, open-source framework used to store and process very large datasets across a cluster of computers. Instead of depending on one powerful machine, Hadoop spreads both the data and the processing over many ordinary machines that work together as a single system.

Why Hadoop is used for large datasets

- **Scalability:** More machines can be added to the cluster when the data grows.
- **Fault tolerance:** Data is stored as blocks that can be replicated, so the failure of one machine does not cause data loss.
- **Cost-effective:** It runs on low-cost commodity hardware.
- **Parallel processing:** Computation is moved to the place where the data is stored, which makes processing of huge datasets faster.
- **Flexibility:** It can store structured data (such as CSV files) as well as semi-structured and unstructured data.

Core components of Hadoop

- **HDFS (Hadoop Distributed File System):** The storage layer. It splits files into blocks and stores them across the nodes of the cluster.
- **YARN (Yet Another Resource Negotiator):** The resource-management layer. It allocates CPU and memory to the applications running in the cluster and schedules jobs.
- **MapReduce:** The processing model. A job is divided into a *Map* phase, which processes the data in parallel, and a *Reduce* phase, which combines the results.

In this practical only the HDFS storage layer was used; YARN and MapReduce were not run.

3. HDFS COMPONENTS

HDFS follows a master-worker architecture. The following daemons were seen running in this practical when the `jps` command was executed.

3.1 NameNode

The NameNode is the **master** of HDFS. It stores the metadata of the file system, such as the directory structure, file names, permissions and the locations of each file's blocks. It does not store the actual data. When a client wants to read or write a file, it first contacts the NameNode.

3.2 DataNode

The DataNode is the **worker** of HDFS. It stores the actual data blocks on its local disk. It serves read and write requests from clients and regularly sends heartbeat and block reports to the NameNode to show that it is alive and working.

3.3 SecondaryNameNode

The SecondaryNameNode helps the NameNode by periodically merging the file-system image (fsimage) with the edit logs (checkpointing). This keeps the edit log small and the NameNode restart time short. It is a helper process and is **not** a backup or standby NameNode.

4. DATASET DESCRIPTION

Dataset name: supermarket_sales.csv

The dataset is a small comma-separated (CSV) file containing sales transactions of a supermarket. It has **one header line and 10 records** (11 lines in total, confirmed using `wc -l`). Each record represents one invoice with its branch, city, product line, quantity sold, price per unit and total amount. The file size is 578 bytes.

Fields in the dataset

Column	Description
Invoice_ID	Unique invoice number of the sale (INV001 to INV010).
Branch	Supermarket branch code (A, B or C).
City	City where the branch is located (Yangon, Mandalay or Naypyitaw).
Product_Line	Category of the product sold.
Quantity	Number of units sold in the invoice.
Unit_Price	Price of one unit.
Total	Total amount of the invoice (Quantity x Unit_Price).

Data stored in the file

Invoice_ID	Branch	City	Product_Line	Qty	Unit_Price	Total
INV001	A	Yangon	Health and Beauty	5	74.50	372.50
INV002	B	Mandalay	Electronic Accessories	3	15.20	45.60
INV003	C	Naypyitaw	Home and Lifestyle	4	30.00	120.00
INV004	A	Yangon	Sports and Travel	2	45.00	90.00
INV005	B	Mandalay	Food and Beverages	6	12.50	75.00
INV006	C	Naypyitaw	Fashion Accessories	3	25.00	75.00
INV007	A	Yangon	Electronic Accessories	5	20.00	100.00
INV008	B	Mandalay	Health and Beauty	2	50.00	100.00
INV009	C	Naypyitaw	Food and Beverages	8	10.00	80.00
INV010	A	Yangon	Home and Lifestyle	4	35.00	140.00

5. HADOOP INSTALLATION / SETUP

Hadoop version **3.4.3** was used, installed in the directory `/home/aiml/hadoop` on an Ubuntu Linux system (the installation archive `hadoop-3.4.3.tar.gz` is present in the home directory). The general installation and configuration process is as follows:

Step 1: Install Java. Hadoop is written in Java, so a JDK must be installed and the `JAVA_HOME` environment variable must be set.

Step 2: Download and extract Hadoop. The Hadoop tar.gz file is extracted and the folder is placed in the home directory.

Step 3: Set environment variables. `HADOOP_HOME` and the Hadoop `bin` and `sbin` folders are added to the `PATH` (usually in the `.bashrc` file) so that commands such as `hadoop`, `hdfs` and `stop-dfs.sh` can be run from any location.

Step 4: Configure Hadoop. The configuration files are edited: `hadoop-env.sh` (Java path), `core-site.xml` (default file-system address), `hdfs-site.xml` (replication factor and NameNode/DataNode directories). For a single-node (pseudo-distributed) setup the replication factor is 1.

Step 5: Set up passwordless SSH. Hadoop's start and stop scripts use SSH to connect to localhost.

Step 6: Format the NameNode. The command `hdfs namenode -format` is run once to initialise the HDFS metadata directory.

Step 7: Start the HDFS services. The command `start-dfs.sh` starts the NameNode, DataNode and SecondaryNameNode.

Step 8: Verify. The `hadoop version` and `jps` commands are used to confirm the installation and that the daemons are running.

In this practical the installation was already completed, and the HDFS daemons were already running when the practical began, as confirmed by the `jps` output shown in Section 9.

6. COMMANDS USED

The commands below are listed in the order in which they were executed in the practical.

Command 1

```
cd ~
```

Explanation: Changes the current directory to the home directory of the user (/home/aiml).

Command 2

```
pwd
```

Explanation: Prints the full path of the present working directory. Here it displays /home/aiml.

Command 3

```
hadoop version
```

Explanation: Displays the installed Hadoop version and build details. It confirms that Hadoop 3.4.3 is installed and working.

Command 4

```
jps
```

Explanation: Lists the Java processes that are running. It is used to check whether the Hadoop daemons (NameNode, DataNode and SecondaryNameNode) are running.

Command 5

```
nano supermarket_sales.csv
```

Explanation: Opens the nano text editor to create the file supermarket_sales.csv in the home directory. The header line and the 10 sales records are typed and saved.

Command 6

```
ls
```

Explanation: Lists the files and folders in the current directory. It is used to confirm that supermarket_sales.csv was created.

Command 7

```
cat supermarket_sales.csv
```

Explanation: Displays the contents of the local CSV file on the terminal so that the data can be checked before uploading.

Command 8

```
hdfs dfs -mkdir -p /supermarket
```

Explanation: Creates the directory /supermarket in HDFS. The -p option creates the directory (and any missing parent directories) without an error if it already exists.

Command 9

```
hdfs dfs -ls /
```

Explanation: Lists the contents of the HDFS root directory. It shows the directories present in HDFS, including the new /supermarket directory.

Command 10

```
hdfs dfs -put supermarket_sales.csv /supermarket/
```

Explanation: Uploads (copies) the local file supermarket_sales.csv into the HDFS directory /supermarket/.

Command 11

```
hdfs dfs -ls /supermarket
```

Explanation: Lists the files inside the HDFS directory /supermarket. It shows the file permissions, replication factor, size and modification time.

Command 12

```
hdfs dfs -cat /supermarket/supermarket_sales.csv
```

Explanation: Displays the contents of the file stored in HDFS.

Command 13

```
hdfs dfs -cat /supermarket/supermarket_sales.csv | wc -l
```

Explanation: Reads the HDFS file and passes it to wc -l, which counts the number of lines. The result 11 means 1 header line plus 10 records.

Command 14

```
hdfs dfs -cat /supermarket/supermarket_sales.csv | grep "Yangon"
```

Explanation: Reads the HDFS file and uses grep to display only the lines that contain the word Yangon. This is used to search records of one city.

Command 15

```
hdfs dfs -cat /supermarket/supermarket_sales.csv | grep "Food and Beverages"
```

Explanation: Displays only the lines that contain the product line Food and Beverages.

Command 16

```
hdfs dfs -cat /supermarket/supermarket_sales.csv | tail -n +2 | cut -d',' -f5 | awk '{sum += $1} END {print "Total Quantity:", sum}'
```

Explanation: Calculates the total quantity sold. Each part of the pipeline does one job: **tail -n +2** skips the header line; **cut -d',' -f5** extracts the 5th comma-separated field (Quantity); **awk** adds all the values into the variable sum and prints it at the end.

Command 17

```
hdfs dfs -du -h /supermarket/supermarket_sales.csv
```

Explanation: Shows the size of the file in HDFS in human-readable form. The first number is the file size and the second is the disk space used including replicas.

Command 18

```
hdfs dfs -cp /supermarket/supermarket_sales.csv /supermarket/supermarket_backup.csv
```

Explanation: Copies the file inside HDFS to create a backup file named supermarket_backup.csv.

Command 19

```
hdfs dfs -ls /supermarket
```

Explanation: Lists the directory again to confirm that both the original file and the backup file are present (2 items).

Command 20

```
hdfs dfs -get /supermarket/supermarket_sales.csv ~/supermarket_download.csv
```

Explanation: Downloads the file from HDFS to the local home directory and saves it as supermarket_download.csv.

Command 21

```
ls
```

Explanation: Lists the local files again to confirm that supermarket_download.csv has been downloaded.

Command 22

```
stop-dfs.sh
```

Explanation: Stops all the HDFS daemons (NameNode, DataNode and SecondaryNameNode).

Command 23

```
jps
```

Explanation: Run again after stopping HDFS. Only the Jps process itself is listed, which confirms that the Hadoop daemons have stopped.

7. STEP-BY-STEP PROCEDURE

Step 1: Check the working location. The terminal was opened and the command `cd ~` was used to go to the home directory. The command `pwd` confirmed the location as `/home/aiml`.

Step 2: Verify Hadoop. The command `hadoop version` showed that Hadoop 3.4.3 is installed.

Step 3: Check the running services. The command `jps` showed that NameNode, DataNode and SecondaryNameNode were running, so HDFS was ready to use.

Step 4: Create the CSV file. The command `nano supermarket_sales.csv` was used to open the editor. The header line (Invoice_ID, Branch, City, Product_Line, Quantity, Unit_Price, Total) and the 10 sales records were typed, and the file was saved. The command `ls` confirmed that the file existed, and `cat supermarket_sales.csv` displayed its contents.

Step 5: Create an HDFS directory. The command `hdfs dfs -mkdir -p /supermarket` created the directory. The command `hdfs dfs -ls /` listed 5 items in HDFS: `/car_sales`, `/online_shopping`, `/supermarket`, `/tmp` and `/user`.

Step 6: Upload the file to HDFS. The command `hdfs dfs -put supermarket_sales.csv /supermarket/` uploaded the file. The command `hdfs dfs -ls /supermarket` showed 1 item of size 578 bytes.

Step 7: View the file in HDFS. The command `hdfs dfs -cat /supermarket/supermarket_sales.csv` displayed the uploaded data, which matched the local file.

Step 8: Count the records. The HDFS file was piped to `wc -l`, which returned 11 (header + 10 records).

Step 9: Search the data. The file was piped to `grep "Yangon"` (4 records found) and to `grep "Food and Beverages"` (2 records found).

Step 10: Analyse the data. A pipeline using `tail`, `cut` and `awk` calculated the total quantity, which was 42.

Step 11: Check the file size. The command `hdfs dfs -du -h` showed the file size as 578.

Step 12: Create a backup. The command `hdfs dfs -cp` created `supermarket_backup.csv` inside `/supermarket`. Listing the directory showed 2 items of 578 bytes each.

Step 13: Download the file. The command `hdfs dfs -get` copied the file from HDFS to the home directory as `supermarket_download.csv`, which was confirmed with `ls`.

Step 14: Verify and stop HDFS. The directory listing and file contents were checked once more. Then `stop-dfs.sh` stopped the NameNode, DataNode and SecondaryNameNode, and `jps` showed only the Jps process.

8. DATA ANALYSIS

8.1 Searching by city

The HDFS file was read with `hdfs dfs -cat` and the output was sent through `grep "Yangon"`. The `grep` command prints only the lines that contain the given text. Four records belong to Yangon: INV001, INV004, INV007 and INV010 (all from Branch A).

8.2 Searching by product category

In the same way, `grep "Food and Beverages"` was used. Two records belong to this product line: INV005 (Mandalay, Branch B) and INV009 (Naypyitaw, Branch C).

8.3 Calculating the total quantity

The total quantity was calculated by the pipeline `tail -n +2 | cut -d',' -f5 | awk`. The `tail` command removed the header line so that only numeric data remained, `cut` picked the Quantity column (field 5), and `awk` added the values one by one. The values 5 + 3 + 4 + 2 + 6 + 3 + 5 + 2 + 8 + 4 give the result **Total Quantity: 42**.

8.4 Calculating the total sales

The total sales amount is the sum of the **Total** column (field 7) of all 10 invoices:

$372.50 + 45.60 + 120.00 + 90.00 + 75.00 + 75.00 + 100.00 + 100.00 + 80.00 + 140.00 = \mathbf{1198.10}$

Note: this total sales value was calculated from the dataset values shown in the practical. It was not printed as a separate command output in the terminal, whereas the total quantity (42) was.

8.5 Summary of calculated values

Item	Value	How obtained
Lines in file (header + records)	11	wc -l (command output)
Number of data records	10	11 lines minus 1 header line
Records for Yangon	4	grep "Yangon" (command output)
Records for Food and Beverages	2	grep (command output)
Total Quantity	42	awk (command output)
Total Sales (sum of Total column)	1198.10	Calculated from the dataset
File size in HDFS	578 bytes	hdfs dfs -ls and -du (command output)

9. OUTPUT / RESULTS

9.1 Hadoop version (hadoop version)

```
Hadoop 3.4.3
Source code repository https://github.com/apache/hadoop.git -r
9d50c6884666e794e45102260a4017bb31802e1b
Compiled by stevel on 2026-02-13T14:23Z
Compiled on platform linux-x86_64
Compiled with protoc 3.23.4
From source with checksum 2331238c4c2929e66645316a32a8613
This command was run using /home/aiml/hadoop/share/hadoop/common/hadoop-common-3.4.3.jar
```

Meaning: Hadoop version 3.4.3 is installed and working. The last line shows the location of the Hadoop jar file inside /home/aiml/hadoop.

9.2 Running daemons (jps)

```
13201 NameNode
13639 SecondaryNameNode
13401 DataNode
15100 Jps
```

Meaning: The three HDFS daemons were running before the practical started. The number at the start of each line is the process ID.

9.3 HDFS root directory (hdfs dfs -ls /)

```
Found 5 items
drwxr-xr-x - aiml supergroup      0 2026-10-01 12:16 /car_sales
drwxr-xr-x - aiml supergroup      0 2026-09-25 15:54 /online_shopping
drwxr-xr-x - aiml supergroup      0 2026-10-01 12:52 /supermarket
drwxr-xr-x - aiml supergroup      0 2026-09-25 15:55 /tmp
drwxr-xr-x - aiml supergroup      0 2026-09-25 15:55 /user
```

Meaning: HDFS contains 5 directories. The newly created /supermarket directory appears in the list. The letter **d** at the start of each line shows that it is a directory, and the owner is aiml.

9.4 File uploaded to HDFS (hdfs dfs -ls /supermarket)

```
Found 1 items
-rw-r--r-- 1 aiml supergroup      578 2026-10-01 12:53 /supermarket/supermarket_sales.csv
```

Meaning: The file was uploaded successfully. The value **1** is the replication factor and **578** is the file size in bytes. The **-** at the start shows that it is a file.

9.5 File contents in HDFS (hdfs dfs -cat)

```
Invoice_ID,Branch,City,Product_Line,Quantity,Unit_Price,Total
INV001,A,Yangon,Health and Beauty,5,74.50,372.50
INV002,B,Mandalay,Electronic Accessories,3,15.20,45.60
INV003,C,Naypyitaw,Home and Lifestyle,4,30.00,120.00
INV004,A,Yangon,Sports and Travel,2,45.00,90.00
INV005,B,Mandalay,Food and Beverages,6,12.50,75.00
INV006,C,Naypyitaw,Fashion Accessories,3,25.00,75.00
INV007,A,Yangon,Electronic Accessories,5,20.00,100.00
INV008,B,Mandalay,Health and Beauty,2,50.00,100.00
INV009,C,Naypyitaw,Food and Beverages,8,10.00,80.00
INV010,A,Yangon,Home and Lifestyle,4,35.00,140.00
```

Meaning: The data stored in HDFS is identical to the local file, so the upload was correct.

9.6 Line count (wc -l)

```
11
```

Meaning: The file has 11 lines: 1 header line and 10 data records.

9.7 Search for Yangon (grep "Yangon")

```
INV001,A,Yangon,Health and Beauty,5,74.50,372.50
INV004,A,Yangon,Sports and Travel,2,45.00,90.00
INV007,A,Yangon,Electronic Accessories,5,20.00,100.00
INV010,A,Yangon,Home and Lifestyle,4,35.00,140.00
```

Meaning: Four sales records were made in Yangon, all in Branch A.

9.8 Search for Food and Beverages (grep "Food and Beverages")

```
INV005,B,Mandalay,Food and Beverages,6,12.50,75.00
INV009,C,Naypyitaw,Food and Beverages,8,10.00,80.00
```

Meaning: Two sales records belong to the Food and Beverages product line.

9.9 Total quantity (tail | cut | awk)

```
Total Quantity: 42
```

Meaning: A total of 42 units were sold across all 10 invoices.

9.10 File size (hdfs dfs -du -h)

```
578 578 /supermarket/supermarket_sales.csv
```

Meaning: The file size is 578 bytes. As the replication factor is 1, the space used on disk is also 578 bytes.

9.11 Backup created (hdfs dfs -cp, then ls /supermarket)

```
Found 2 items
-rw-r--r-- 1 aiml supergroup 578 2026-10-01 12:56 /supermarket/supermarket_backup.csv
-rw-r--r-- 1 aiml supergroup 578 2026-10-01 12:53 /supermarket/supermarket_sales.csv
```

Meaning: The backup file has the same size (578 bytes) as the original, so the copy was successful.

9.12 File downloaded from HDFS (hdfs dfs -get, then ls - relevant entries only)

```
supermarket_download.csv  supermarket_sales.csv
```

Meaning: The file supermarket_download.csv now appears in the local home directory, showing that the file was downloaded from HDFS successfully.

9.13 Stopping HDFS (stop-dfs.sh, then jps)

```
Stopping namenodes on [localhost]
Stopping datanodes
Stopping secondary namenodes [aiml-D204-75]
16941 Jps
```

Meaning: All three HDFS daemons were stopped. Only the Jps process remains in the list, which confirms that HDFS is no longer running.

10. CONCLUSION

In this practical, the Hadoop 3.4.3 installation was verified and the HDFS daemons (NameNode, DataNode and SecondaryNameNode) were checked using `jps`. A supermarket sales CSV file was created locally, uploaded to the HDFS directory `/supermarket`, and viewed using HDFS shell commands.

The data was searched with `grep` (4 records for Yangon and 2 records for Food and Beverages) and analysed with `tail`, `cut` and `awk` to obtain a total quantity of 42. File management operations such as `-du`, `-cp` and `-get` were also performed, and HDFS was stopped properly using `stop-dfs.sh`.

Through this practical, the basic working of HDFS, the roles of its main components, and the use of HDFS commands together with Linux utilities for simple data analysis were understood.